

Problem 1 · Talk script (spoken version)

About 2 min 30 s, 7 slides. Deck:

[docs/PPT/problem1_speed_and_path_tracking_difficulty.pptx](#) Rewrite anything that does not flow when you say it out loud — this version was written to be spoken.

P1 · Cover

I will talk about Problem 1 — why path tracking gets harder the faster the car goes.

I used three controllers — pure pursuit, LQR and MPC — and ran them on a 12-metre-radius circle at 3, 5 and 7 m/s, nine runs in total. **P2 · Three causes**

I will give the conclusion first. Getting harder at speed is not caused by one thing — three things come together, and they differ a lot in magnitude.

First, the lateral acceleration a corner demands is v^2 times the curvature. Going from 3 to 7 m/s takes that demand from 0.75 to 4.08 — more than a factor of five. There is no way around this one; it is physics.

Second, and most important: the look-ahead distance grows together with speed, and pure pursuit's corner cut is exactly proportional to that look-ahead distance. I fitted that relationship from 13 runs; R square is 0.9988, essentially a straight line.

Third, the control period is fixed. The faster the car goes, the further it travels per step, so the number of corrections available in a given corner falls off as one over v .

One point I want to stress here: it is not that steering runs out. Steady-state steer depends only on curvature, not on speed, and even at high speed it uses just 38% of the limit — there is plenty of steering margin left.

P3 · Experimental setup

The experiment itself is simple. The car has a 2.5 m wheelbase, the front wheels go to at most 35 degrees, and the steering rate is at most 45 degrees per second.

I split the effect of speed into three quantities and the conclusion is very clean.

The steering angle does not move — across the three speeds it is basically a horizontal line, because it depends only on wheelbase and curvature.

Yaw rate grows linearly, from 0.26 to 0.53 — a little over a factor of two.

Normal acceleration grows with the square, from 0.75 to 3.17 — more than four times — and it sits right on the theoretical $v^2\kappa$ line, within 3.4%.

So what speed really changes is not "how many degrees must I turn" but "how much lateral drift those degrees correspond to".

P4 · Main cause

This is the most important slide in the whole talk.

When pure pursuit runs the circle, it does not sit on the reference circle — it settles onto a slightly smaller concentric circle.

At 3 m/s it drifts 0.437 m inwards. But the theoretical steer needed there is only 0.205 rad, so steering is nowhere near saturation. So where do those 0.44 m come from?

I ran several controls, changing one thing at a time: only the look-ahead distance, only the front/rear axle split, only the resampling spacing.

In the end I fitted 13 points: the cut is roughly 0.93 times the look-ahead distance times the sideslip angle.

The most convincing one is the second group. I set the front/rear axle split to 2.5 and 0, forcing the sideslip angle β to exactly zero — and the cut immediately became -0.042 m, essentially gone.

So the cut is caused by β . Speed enters that chain through the look-ahead distance: higher speed, longer look-ahead, larger cut.

P5 · Error comparison

The peak steady-state lateral error of all three controllers rises with speed. At the high-speed setting they are 0.53, 0.48 and 0.35 m. MPC is best, LQR in the middle, pure pursuit worst.

But there is a trap here that I should point out. If you look at the whole-run mean error, the conclusion flips — it goes down, with pure pursuit dropping from 0.286 to 0.275.

Why? Because as speed rises the trajectory contains more samples on the straight sections. Straight-section error is always zero, and that drags the mean down.

So to judge whether tracking is hard, you have to look at the steady segment and the peak value, not the whole-run mean.

P6 · Conclusions and recommendations

To sum up. Getting harder at speed is three costs stacked on top of each other: a quadratic physical cost, a linear geometric cost, and a timing cost that falls off as $1/v$. The dominant issue is still pure pursuit's corner cut.

Two recommendations.

First, do not scale the look-ahead distance blindly with speed — pair it with curvature feed-forward or a steering-offset compensation. LQR and MPC are accurate precisely because they use curvature explicitly.

Second, when decelerating near the goal, trigger it on the remaining arc length along the path, not on straight-line distance, otherwise the car will slow down in the middle of the run for no reason.

Appendix 1 · Questions the examiner may ask

Where does the corner-cut formula come from? It was fitted from 13 controlled runs, not derived first and then verified. The strongest supporting evidence is the $\beta = 0$ control — setting the front/rear axle split to (2.5, 0) drives the cut to zero outright.

Why not use the classical $Lf^2\kappa/8$ expression? It underestimates by a factor of 1.7 to 2.6, and it predicts that the cut does not depend on β — which the $\beta = 0$ control rules out directly.

Why is MPC the most accurate? It writes the curvature straight into the optimisation objective; LQR adds it as a feed-forward term; pure pursuit has none of that and can only correct after the fact through feedback.

Appendix 2 · If asked "what is still not done"

1. `lqr_dynamic` (dynamic LQR) is not implemented; this work only has the kinematic LQR.
2. Only the `circle` route was tested — `double_lane_change`, `right_angle`, `s_curve` and `mixed_course` were not run, so I will not claim that the conclusion holds on other routes.
3. Cause ③ (further travel per step) is intuitively right, but **this experiment did not isolate and verify it** — say so honestly when presenting.

Problem 4 · Talk script (spoken version)

About 3 min 30 s, 8 slides. Deck: [docs/PPT/problem4_dorm_to_classroom_navigation.pptx](#)
 Rewrite anything that does not flow when you say it out loud — this version was written to be spoken.

P1 · Cover

I will report on Problem 4: navigating from the dormitory to the classroom. I did four things — build the map, generate the reference path and assign speeds to it, track it with pure pursuit, and then run experiments and look at the data.

P2 · Task pipeline

This is the whole pipeline. The four stages are map construction, reference path generation, pure-pursuit implementation and experimental data analysis; the output of each is the input of the next. Everything runs offline, with no network. The clip below is the first-person view of the car driving the full reference path, 1450.6 metres.

P3 · Map construction

The map uses raw OSM data. I extracted the drivable roads around campus into a road network: 1745 nodes, 1989 edges, 146.7 km total length, and the whole graph is connected. The start and goal are first snapped to the nearest nodes, 44.4 m and 74.5 m off respectively, then Dijkstra is run, giving a 1463.5 m polyline.

P4 · Reference path

This is where the biggest problem was. Taken as-is, the polyline has a peak curvature of 1.6049, a radius of only 0.62 m. With a 35-degree front-wheel limit and a 2.5 m wheelbase, this car's curvature limit is 0.2801, corresponding to a 3.57 m radius — the raw polyline exceeds it by 5.7 times and is physically impossible to drive. I did two things: Gaussian smoothing of the curvature first, then filleting the corners. Curvature came down to 0.1166. Smoothing accounted for 92% of the reduction and filleting only 14%.

P5 · Speed profile

With the curvature in hand I can assign speeds: v equals the square root of the lateral-acceleration limit divided by the curvature. On the same route, the peak lateral acceleration is 7.30 at constant speed and drops to 4.34 under the curvature-based limit. I hit one pitfall here: the speed limit must be propagated point by point — scan the deceleration sections backwards and the acceleration sections forwards. I first wrote it as a one-shot array operation where each point only looks at itself, and the car simply would not slow down.

P6 · Look-ahead distance and speed

This page sweeps two parameters: the look-ahead distance from 1 to 12 m and the speed from 3 to 8 m/s, 18 simulations in total. As the look-ahead grows, the peak error rises from 1.06 to 4.12 m, but the steering chatter falls from 23.8 to 1.8 degrees per second — you can only have one of line-following and smoothness. Raising the speed also raises the error: from 3 to 7 m/s the peak error goes from 1.42 to 1.64 m.

P7 · Conclusions

Four conclusions. One: curvature smoothing has the largest effect — same car, same speed, adding smoothing alone takes the peak lateral acceleration from 17.83 down to 7.97. Two: the curvature-based speed limit pushes the lateral acceleration down from 7.30 to 4.34, at the cost of 0.31 m more terminal error. Three: the higher the speed, the larger the error — from 3 to 7 m/s the peak error rises from 1.42 to 1.64 m. Four: the look-ahead distance has to be a compromise. There is also a detail that stuck with me: the peak error of all 18 simulations falls at the same place — the sharp bend at about $s = 589$ m with a 2.4 m radius. The tracking-quality ceiling for the whole route is set by that one corner.

P8 · Closing

That are our report. Thank you.